



City University of Hong Kong
Department of Computer Science

Final Year Project Report

Zhitao Zhu

Supervisor: Dong Minjing

Bachelor of Science in *Computer Science*

January 24, 2026

Abstract

Urban traffic congestion remains a critical challenge in current years, driven by the reliance on inefficient fixed-time traffic signal control systems. This project addresses this issue by developing an integrated Intelligent Traffic Control and Surveillance System within a simulated environment. The core of the system is a Deep Q-Network (DQN) reinforcement learning agent that dynamically controls a four-way intersection simulated in SUMO (Simulation of Urban MObility) via the TraCI API. To ensure real-world relevance, the system incorporates a synthetic vehicle detection module that minimize the noise and delay of YOLO-style perception models, providing realistic input for the RL agent. The DQN controller's performance is evaluated against traditional fixed-time and rule-based adaptive strategies. Crucially, the evaluation expands beyond some typical metrics (waiting time and queue length) to include environmental impact metrics, such as fuel emissions, which could extracted directly from SUMO's emission models. The resulting system is demonstrated through a real-time Streamlit dashboard, proving the efficacy of RL in creating a safer, more efficient, and environmentally sustainable urban traffic solution.

Keywords:

Acknowledgements

I would like to express my deep gratitude to Professor Dong Minjing my research supervisor, for their patient guidance, enthusiastic encouragement and useful critiques of this research work. I would also like to thank Ms. Christy CHAN, for her advice and assistance in keeping my progress on schedule.

Finally, I wish to thank my parents for their support and encouragement throughout my study.

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem statement	2
1.3	Problem definition and Scope	2
1.4	Aims and objectives	4
1.5	Organization of the report	5
2	Literature Review	6
2.1	Reinforcement Learning for Adaptive Traffic Signal Control (TSC) . . .	7
2.2	From Q-Learning to Deep RL	8
2.3	Baseline Methods for Traffic Signal Control	8
2.4	Traffic Control and Management with Simulation	9
3	Methodology	11
3.1	System Architecture and Simulation Environment	11
3.2	SUMO: The Microscopic Traffic Simulator	13
3.3	TraCI: The Control Interface	13
3.4	Fixed-Time Controller(FTC)	14
3.5	Rule-Based Adaptive Controller(RAC)	14
3.5.1	Real-Time Demand Score Calculation	14
3.5.2	Dynamic Decision Logic	15
3.6	Deep Q-Network (DQN) Controller	16

<i>CONTENTS</i>	iv
3.7 Performance Evaluation Metric	20
3.8 Visualization Dashboard	21
4 Results	22
4.1 section a	22
4.2	22
4.3 Summary	22
5 Discussion	23
6 Conclusions	24
References	25
Appendices	30
A A	30

Chapter 1

Introduction

1.1 Background

The rapid expansion of urban centers has led to an exponential increase in vehicle ownership, resulting in pervasive traffic congestion worldwide. This congestion is not merely an inconvenience; it leads to significant economic losses through wasted time and fuel, increases the risk of traffic accidents, and contributes substantially to harmful environmental pollution [1]. Furthermore, modern urban planning requires a shift toward systems that minimize vehicle idling and the resulting emissions to meet sustainability goals [2].

The primary technological bottleneck lies in traditional traffic light systems, which most cities still rely on. These systems often operate on pre-defined, fixed timers and cannot adapt to real-time traffic conditions [3]. While Adaptive Traffic Signal Control (ATSC) and Reinforcement Learning (RL) have shown promise in reducing delays [4], existing research frequently suffers from limitations. Many studies assume idealized settings, evaluate algorithms on narrow, single-intersection scenarios, and lack a unified platform for fair comparison against strong baselines [5].

A critical gap remains in the scalability of these controls. Real-world city networks consist of heterogeneous intersections with varying phase programs and topologies[6]. A

solution optimized for a single isolated junction rarely scales effectively to a network level, where actions taken at one node impact the inflow of another [7]. To bridge this gap, this project proposes a comprehensive simulation and benchmarking platform. By leveraging OpenStreetMap (OSM) data, the system models realistic network topologies—providing the heterogeneity necessary for robust evaluation.

1.2 Problem statement

The core problems addressed in this project can be summarized as follows:

1. Inefficiency of Static Control:

Traditional fixed-time signal controllers rely on historical data and cannot adapt to real-time demand variations, resulting in unnecessary waiting times and increased emissions[3].

2. Need for Fair Benchmarking

The field lacks standardized evaluation environments. Many proposed algorithms are not evaluated under consistent conditions, making it difficult to benchmark the true performance of RL agents against established heuristic methods[5].

3. Scalability Challenge:

Methods are often tested only on "toy" intersections. Validating control strategies requires testing on multi-intersection networks with realistic, diverse Traffic Light System (TLS) programs[8, 9].

1.3 Problem definition and Scope

This project focuses on the development and evaluation of intelligent traffic signal control strategies within a microscopic simulation environment.

1. **Environment:** The project utilizes the SUMO (Simulation of Urban Mobility) traffic simulator, controlled externally via the TraCI Python API[10].
2. **Controlled Assets:**
 - (a) Custom Dataset: A controlled, single-intersection environment used for debugging and initial logic verification.
 - (b) OSM Datasets: Realistic map networks imported from OpenStreetMap (e.g., Cologne, Vancouver). The system controls a selected subset of traffic lights, utilizing dynamic TLS detection to identify controllable nodes.
3. **Observations (State Space):** Real-time metrics including per-lane vehicle counts, queue lengths, average speeds, and current TLS phases.
4. **Actions:**
 - (a) **Baselines:** Phase switching or green extension logic based on heuristic rules (e.g., pressure, gaps).
 - (b) **MARL Agent:** A binary decision per TLS: "Keep current phase" or "Switch to next phase" (2-action interface).
5. **Objectives:** Minimize average waiting time and queue length. In addition, assess environmental impact via emissions and fuel consumption metrics (derived from SUMO emission models [11]).
6. **Constraints:** The system must handle heterogeneous TLS phase definitions inherent in OSM imports and operates under computational limits necessitating "headless" simulation for large maps.
7. **Out of Scope:** Real-world hardware deployment, sensor calibration, and simultaneous control of all signals in a massive city-wide network (scope is limited to selected sub-networks).

1.4 Aims and objectives

The primary aim of this project is to create a robust, reproducible platform for benchmarking traffic signal control strategies and to demonstrate the efficacy of Multi-Agent Reinforcement Learning in realistic environments. The specific objectives are:

1. Develop a reproducible simulation platform: Build a system using SUMO [5] and TraCI that supports both custom single-intersections and complex, multi-intersection networks derived from OSM.
2. Implement strong baselines for benchmarking: Integrate and standardize multiple non-RL strategies to establish a fair performance baseline, including:
 - (a) Fixed-Time Control
 - (b) Rule-Based Adaptive Control
 - (c) MaxPressure Control[12]
 - (d) Self-Organizing Traffic Light Control (SOTL)[13]
 - (e) Genetic Algorithm (GA) optimization[14]
3. Implement a MARL controller: Develop a Multi-Agent RL controller using parameter sharing (Shared DQN) capable of controlling multiple heterogeneous traffic lights[15].
4. Build a Web-Based Experiment Interface: Develop a Flask-based user interface to facilitate experiment configuration, simulation launching, and result visualization
5. Conduct comparative analysis: Perform experiments to evaluate the strategies, identifying performance gains and limitations regarding traffic efficiency and environmental impact[2].

Table 1.1: Template of structure

Front	Abstract	
	Acknowledgements	
	Table of Contents	
Main text	Chapter 1	Introduction
	Chapter 2	Literature Review
	Chapter 3	Methodology
	Chapter 4	Results
	Chapter 5	Discussions
	Chapter 6	Conclusions
End	References	
	Appendix	

1.5 Organization of the report

Table 1.1 shows the overall structure of the report. This report is organized into six chapters. The report is organized as follows: Chapter 2 reviews existing work in traffic signal control and RL [4, 16]. Chapter 3 details the system architecture and Flask interface design. Chapter 4 describes the methodology, including the MARL algorithm and baseline logic. Chapter 5 outlines the experiment design and testing procedures. Chapter 6 presents preliminary results, and Chapter 7 discusses future work and conclusions.

Chapter 2

Literature Review

The development and validation of advanced traffic signal controllers, particularly those using reinforcement learning, require a reliable and accurate simulation environment. Many traffic simulation tools exist, offering features for traffic flow modeling, connected and autonomous vehicle (CAV) studies, and emissions analysis. Well-known platforms include SimTraffic [10] and AIMSUN [17]; these commercial and research tools provide features suited for different scales of modeling, ranging from microscopic detail to large network analyses.

This study focuses on SUMO (Simulation of Urban MObility) [11], which we chose for its open-source nature and ability to perform detailed microscopic simulations. SUMO is capable of simulating individual vehicle behaviors, including car-following and lane-changing, while also estimating emissions—an important evaluation metric for this work. Its Traffic Control Interface (TraCI) allows external scripts to interact with the simulation, enabling real-time monitoring of traffic states and dynamic signal control, which makes SUMO especially suitable for RL-based research. To enhance realism in multi-intersection scenarios, SUMO supports the import of real-world road networks from OpenStreetMap (OSM) [11], allowing for the simulation of complex urban environments derived from crowdsourced geographic data.

Advancements in adaptive traffic signal control have also led to practical systems

such as the InSync Adaptive Traffic Signal Control System [18] and the Scalable Urban Traffic Control (SURTRAC) framework [19], which leverage artificial intelligence and decentralized optimization to enhance urban mobility. InSync employs image processing to adapt signal timings on a per-second basis, while SURTRAC uses a decentralized, schedule-driven model to coordinate traffic flows across networks. Both systems demonstrate significant improvements over traditional methods, but they rely heavily on high-quality input data and infrastructure.

2.1 Reinforcement Learning for Adaptive Traffic Signal Control (TSC)

Intelligent control techniques have gained substantial momentum in adaptive traffic signal management, with Reinforcement Learning (RL) recognized as an efficient and widely adopted framework [16, 20]. RL is particularly appealing because it is capable of attaining adaptive strategies in complex, high-dimensional environments, with deep neural networks offering the capacity to approximate nonlinear behaviors of the system. Much of the literature highlights notable enhancements in standard traffic performance measures, particularly average delay, travel time, waiting time, and queue length [4].

Despite these successes, the environmental dimension of adaptive signal control is less developed. Noaen et al. [4] reviewed 127 RL-based studies and found that only 19 addressed emissions or energy consumption. While some research shows that reduced travel time can lead to lower fuel use [2], others report trade-offs between traffic efficiency and CO₂ reduction [21], with outcomes depending heavily on traffic scenarios [22].

For multi-intersection networks, Multi-Agent Reinforcement Learning (MARL) extends single-agent RL by modeling each intersection as an independent agent that coordinates with neighbors to achieve global optimization [15, 23]. Recent reviews by Xiao et al. [24] highlight that approaches such as coordinated deep RL have shown su-

perior performance in SUMO simulations, reducing waiting times and enhancing traffic efficiency across interconnected intersections.

2.2 From Q-Learning to Deep RL

The early RL applications in traffic signal control were based on Q-learning [25], initially tested in isolated single intersections [26]. Later works extended the multi-agent framework for handling multiple intersections [7] and larger grids [8], though scalability issues emerged due to the exponential growth of joint action spaces. Methods such as regional partitioning [8] reduced complexity but relied on handcrafted features and linear function approximators, limiting generalization.

This barrier was solved when Mnih et al. [27] introduced deep Q-networks (DQNs), stabilizing training with replay buffers and target networks. DQNs became a benchmark in RL research and were adapted to traffic signal control [9]. Li et al. [28] applied a DQN with stacked autoencoders [29], while Genders and Razavi [9] employed discrete traffic states. These works demonstrated improvements in isolated intersections, but scaling DQN to city-wide systems is computationally demanding, and uncoordinated policies are unlikely to achieve global optimality.

2.3 Baseline Methods for Traffic Signal Control

In addition to RL-based approaches, several non-learning baselines have been widely used for adaptive traffic signal control, serving as benchmarks for evaluating advanced methods.

Max-Pressure (MP) control is a decentralized algorithm that selects signal phases based on the "pressure" difference in upstream and downstream queue lengths, aiming to maximize throughput [12]. Recent enhancements consider phase switching loss to further optimize network performance [30]. MP has shown stability under varying traffic

conditions and is often compared to RL methods in multi-intersection scenarios.

Self-Organizing Traffic Lights (SOTL) represent a rule-based adaptive strategy where signals self-organize based on vehicle counts approaching intersections, forming platoons to minimize stops [13]. History-based variants have also been proposed to improve adaptability [31]. SOTL outperforms rigid timings at low densities by dynamically switching phases to accommodate incoming traffic flows.

Genetic Algorithms (GA) optimize signal timings by evolving populations of timing plans, minimizing objectives like delay or throughput [14]. GA approaches have been applied to oversaturated conditions and provide robust solutions for complex networks, serving as evolutionary baselines against RL.

2.4 Traffic Control and Management with Simulation

Beyond algorithmic improvements, realistic simulation plays a key role in testing and validating adaptive traffic control systems. Microscopic simulators such as SUMO have become the standard for RL-based adaptive traffic research. Unlike macroscopic models, SUMO enables fine-grained simulation of individual vehicle behavior, including lane-changing and car-following, and supports emissions modeling [11].

Several studies illustrate its role in testing robustness and transferability. For example, Tan et al. [32] introduced disturbances (truck events) at intersections, while Wu et al. [33] applied RL to freeway bottlenecks with varying parameters. Han et al. [34] combined macroscopic and microscopic simulations for better realism. Beyond simulation, SUMO has also been integrated with scaled physical testbeds: Jang et al. [35] validated RL-based merging control experimentally, while Chalaki et al. [36] used adversarial learning for real-world policy transfer.

In conclusion, these works demonstrate SUMO's central role in advancing RL-based traffic signal and traffic flow management research. Its detail allows researchers to introduce diverse disturbances, evaluate robustness, and bridge the gap between simulated

and real-world traffic conditions.

Chapter 3

Methodology

This chapter demonstrate the design, implementation, and evaluation processes used to develop the Intelligent Traffic Control and Surveillance System. The methodology is divided into several key stages: system architecture design, simulation environment setup, perception module implementation, control strategy benchmarking, reinforcement learning agent design and performance evaluation.

3.1 System Architecture and Simulation Environment

The foundation of this project is an integrated Intelligent Traffic Control and Surveillance System. Its methodology is built upon a closed-loop system where a Deep Q-Network (DQN) reinforcement learning agent interacts dynamically with a simulated traffic environment. This closed-loop setup allows for rapid, repeatable, and safe testing of the control algorithms.

The overall architecture, illustrated in the diagram [3.1](#) below, is divided into four main functional blocks: the SUMO Environment, the RL Agent, the Replay Buffer, and the DQN Networks. This design implements the standard Deep Q-Learning framework to learn an optimal traffic control policy.

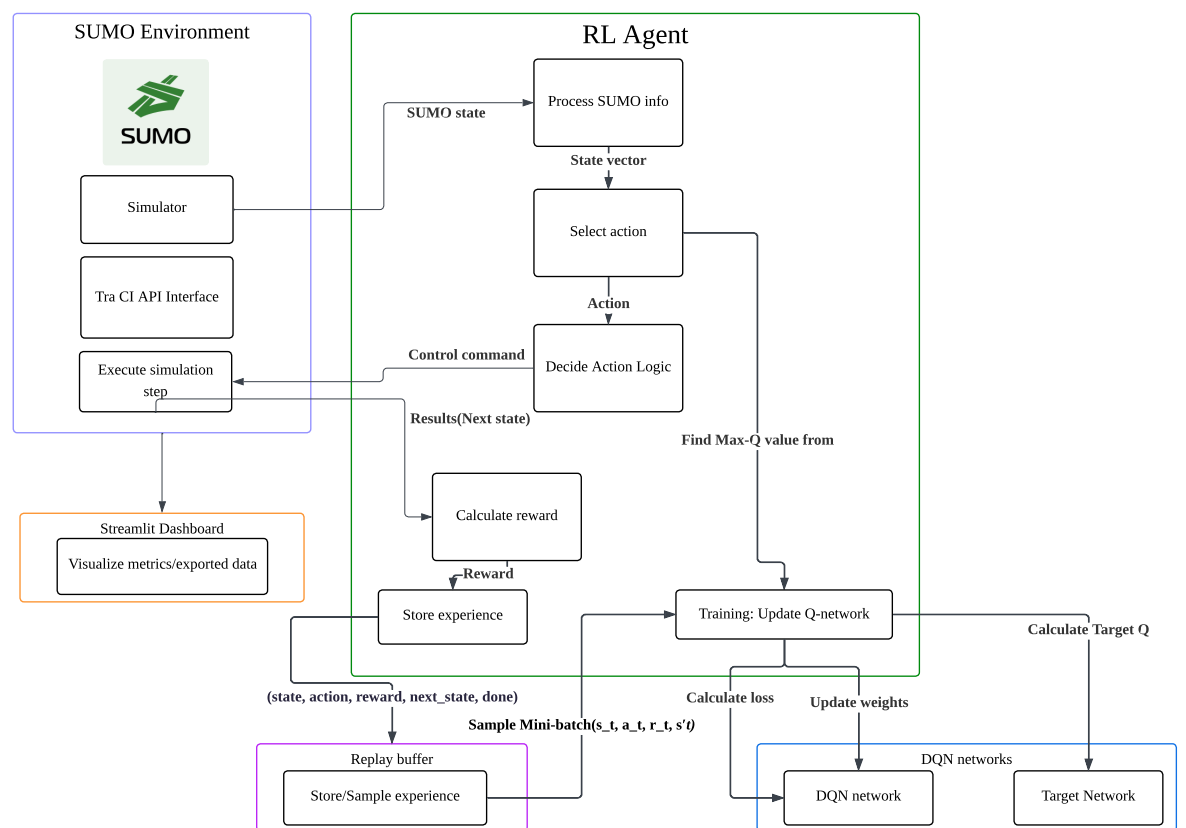


Figure 3.1: System diagram

3.2 SUMO: The Microscopic Traffic Simulator

The simulation environment was built using SUMO (Simulation of Urban Mobility), an open-source, microscopic traffic simulator. We chose SUMO for several key reasons:

- **Microscopic Simulation:** Unlike macroscopic models that treat traffic as a fluid, SUMO simulates the movement and behavior of each individual vehicle, including lane-changing and car-following logic. This high-fidelity modeling is essential for accurately evaluating the impact of signal timing changes.
- **Environmental Modeling:** SUMO includes models for estimating fuel consumption and vehicular emissions (CO_2 , NO_x , etc.) for each vehicle based on its movement profile (acceleration, deceleration, idling). This feature is critical for addressing the project's environmental objectives.
- **Interoperability:** SUMO is designed to be controlled by external applications, making it ideal for reinforcement learning research.

3.3 TraCI: The Control Interface

Real-time interaction between the Python-based control agent and the SUMO simulation is enabled by the Traffic Control Interface (TraCI). TraCI functions as a client-server protocol, allowing the the client to:

- **Retrieve Data (State Observation):** Query the simulation for real-time data at each step, such as the number of vehicles on a lane, their waiting times, speeds, and positions.
- **Execute Commands (Action):** Dynamically change the state of simulation objects, most importantly, setting the traffic light phase for the intersection.

This tight integration allows the reinforcement learning agent to observe the environment, take an action, and receive feedback in a continuous loop.

3.4 Fixed-Time Controller(FTC)

The Fixed-Time Controller (FTC) serves as the non-adaptive benchmark. This strategy operates on a rigid, preset 120-second cycle that is completely independent of real-time traffic conditions. While stable and predictable, this fixed nature is inherently inefficient under dynamic traffic loads. The FTC sequentially cycles through the following four-phase timing plan:

Phase1	East-West Straight Through	30 seconds
Phase2	East-West Straight Through	15 seconds
Phase3	North-South Straight Through	30 seconds
Phase4	North-South Left Turn	15 seconds

The controller advances to the next phase in a simple modulo four sequence

$$\text{Next_phase} = (\text{Current_phase} + 1) \pmod{4}$$

3.5 Rule-Based Adaptive Controller(RAC)

The Rule-Based Adaptive Controller (RAC) is a complex, real-time strategy designed to mitigate congestion by dynamically adjusting the green time duration based on instant traffic demand. The controller operates on a cycle-free, demand-responsive logic centered on a composite demand score.

3.5.1 Real-Time Demand Score Calculation

At each decision step, the RAC calculates a composite demand score for every phase. This score is a weighted average of three primary traffic metrics, each normalized to a range of $[0, 1.0]$. The weights are calibrated to prioritize the length of standing queues, followed by lane occupancy and vehicle speed.

The metrics are calculated as follows:

Queue Length Score (S_{queue}): Weighted at 40

$$S_{queue} = \min \left(1.0, \frac{Que_len}{10.0} \right)$$

Occupancy Score (S_{occ}): Weighted at 30

$$S_{occ} = \min \left(1.0, \frac{Occupancy}{0.8} \right)$$

Speed Score (S_{speed}): Weighted at 30

$$S_{speed} = \max \left(0.0, 1.0 - \frac{Mean_speed}{15.0} \right)$$

Finally, composite demand score is calculated as:

$$D = 0.4 \cdot S_{queue} + 0.3 \cdot S_{occ} + 0.3 \cdot S_{speed}$$

3.5.2 Dynamic Decision Logic

The RAC employs a two-tier decision process: Extension (to maintain the current phase) or Switching (to service a more congested phase).

Option A: Phase Extension The controller favors extending the current phase if its demand remains high while competing phases exhibit minimal congestion. A phase extension is triggered if the current phase's demand score is > 0.6 and the collective demand of all other phases is < 0.4 . The duration of the extension is proportional to the current phase's demand, capped by minimum and maximum limits:

$$\text{extension} = \min(20s, \max(5s, \text{current_demand_score} \times 20))$$

Option B: Phase Switching If the criteria for extension are not met, the controller switches the signal to the phase with the highest instantaneous demand score. The green duration for the newly selected phase is dynamically adjusted from a base time

(base_duration) based on its calculated demand score:

- High Demand ($D > 0.7$): Duration is increased to $1.5 \times \text{base_duration}$.
- Low Demand ($D < 0.3$): Duration is decreased to $0.8 \times \text{base_duration}$.
- Medium Demand (D between 0.3 and 0.7): Duration remains at base_duration

3.6 Deep Q-Network (DQN) Controller

To overcome the limitations of fixed-time and rule-based controllers, the core of this project is an intelligent agent based on Reinforcement Learning (RL), specifically the Deep Q-Network (DQN) algorithm. RL allows an agent to learn an optimal control policy by interacting directly with its environment through trial and error.

The standard Q-learning algorithm, which uses a table to store values for every state-action pair, is infeasible for traffic control due to the virtually infinite number of possible traffic states. The DQN algorithm [27] solves this by using a deep neural network to approximate the action-value function, $Q(s, a)$.

The Markov Decision Process (MDP) Framework

The intersection control problem is modelled as a Markov Decision Process (MDP) defined by:

- **State (s):** A 37-dimensional feature vector representing the current traffic condition, including lane queue lengths, mean speeds, vehicle counts, detector occupancy, and the current active phase (one-hot encoded).
- **Action (a):** A discrete set of eight possible actions, including maintaining the current phase or switching to one of seven alternative phases.
- **Reward (r):** The reward function is the most critical component, as it guides the agent's learning. A complex, multi-objective reward function was designed to

create a balanced, efficient, and stable traffic flow. The total reward R_t at each step t is a weighted sum of five components:

$$R_t = R_{\text{queue}} + R_{\text{speed}} + R_{\text{balance}} + R_{\text{stable}} + R_{\text{occupancy}}$$

Where each component is defined as:

1. **Queue Management (R_{queue}):** This component penalizes the current queue length while rewarding queue reduction.

$$R_{\text{queue}} = -0.1 \cdot Q_{\text{total}} + 0.05 \cdot (Q_{\text{prev}} - Q_{\text{total}})$$

where Q_{total} is the total queued vehicles and Q_{prev} is the total from the previous step.

2. **Speed Management (R_{speed}):** This component rewards higher average speeds and speed improvements.

$$R_{\text{speed}} = 0.02 \cdot V_{\text{avg}} + 0.03 \cdot (V_{\text{avg}} - V_{\text{prev}})$$

where V_{avg} is the current average speed and V_{prev} is from the previous step.

3. **Traffic Balance (R_{balance}):** This component penalizes imbalance between the main traffic directions.

$$R_{\text{balance}} = -0.02 \cdot |N_{\text{EW}} - N_{\text{NS}}|$$

where N_{EW} and N_{NS} are the vehicle counts in the East-West and North-South directions, respectively.

4. **Control Stability (R_{stable}):** This component provides a small reward for

not changing the signal phase, which prevents rapid, inefficient switching.

$$R_{\text{stable}} = \begin{cases} 0.01 & \text{if } a = \text{"keep phase"} \\ 0 & \text{otherwise} \end{cases}$$

5. **Occupancy Control ($R_{\text{occupancy}}$):** This component encourages free-flowing traffic by rewarding low road occupancy and penalizing high occupancy (congestion).

$$R_{\text{occupancy}} = \begin{cases} 0.02 \cdot (0.3 - O_{\text{avg}}) & \text{if } O_{\text{avg}} < 0.3 \\ -0.01 \cdot (O_{\text{avg}} - 0.3) & \text{if } O_{\text{avg}} \geq 0.3 \end{cases}$$

where O_{avg} is the average detector occupancy.

DQN Architecture and Training

- **Network Architecture:** The reinforcement-learning controller is implemented as a deep Q-network implemented in PyTorch and trained offline against episodic SUMO simulations. Its 37-dimensional input vector is constructed from the TrafficState encoder and concatenate eight normalized lane vehicle counts, eight queue-length ratios, eight mean-speed ratios, eight detector occupancies, a four-way one-hot encoding of the active signal phase, and a scalar representing the remaining green time.

This state is propagated through three fully connected layers of 256, 128, and 64 neurons, each leverage ReLU activations and 0.2 dropout to promote generalization across traffic conditions. The final linear layer exposes eight scalar Q-values corresponding to the agent's discrete action set: maintain the current phase, switch to any of the four defined phases, or extend the current phase by 5, 10, or 15 seconds. During training, experience tuples collected from SUMO/TraCI simulation runs are sampled from a replay buffer, target values are computed

via a periodically synchronized target network, and the network is updated by minimizing mean-squared Bellman error using Adam optimization with gradient clipping

- **Training Algorithm:** The agent is trained following the standard DQN framework (Algorithm 1). A replay buffer stores past experiences, and mini-batches are sampled to update the main network through stochastic gradient descent. Target values are computed using the Bellman equation, and the target network is periodically synchronized with the main network. An ϵ -greedy policy is applied to balance exploration and exploitation, where ϵ decays gradually after each episode.

Algorithm 1 Deep Q-Network (DQN) Training Algorithm for Traffic Control

```

1: Initialize: replay memory buffer  $D$  to capacity  $N$ 
2: Initialize: main Q-network  $Q$  with random weights  $\theta$ 
3: Initialize: target network  $\hat{Q}$  with weights  $\hat{\theta} = \theta$ 
4: Initialize: exploration rate  $\epsilon$ , decay rate  $\epsilon_{decay}$ , and minimum  $\epsilon_{min}$ 
5: Initialize: discount factor  $\gamma$  and target network update frequency  $C$ 
6: for episode = 1 to  $M$  do
7:   Reset simulation environment (SUMO) and get initial state  $s_1$ 
8:   for  $t = 1$  to  $T$  do
9:     Select Action:
10:      With probability  $\epsilon$ , select a random action  $a_t$ 
11:      Otherwise, select  $a_t = \arg \max_a Q(s_t, a; \theta)$ 
12:     Execute Action:  $a_t$  in the environment
13:     Observe new state  $s_{t+1}$ 
14:     Calculate Reward  $r_t = \text{CalculateReward}(s_t, s_{t+1}, a_t)$ 
15:     Store transition  $(s_t, a_t, r_t, s_{t+1})$  in replay buffer  $D$ 
16:     Sample a random mini-batch of  $K$  transitions  $(s_j, a_j, r_j, s_{j+1})$  from  $D$ 
17:     Calculate Target  $y_j$  for each transition in the mini-batch:
18:       
$$y_j = \begin{cases} r_j & \text{if episode terminates at step } j + 1 \\ r_j + \gamma \max_{a'} \hat{Q}(s_{j+1}, a'; \hat{\theta}) & \text{otherwise} \end{cases}$$

19:     Calculate Loss  $L = \frac{1}{K} \sum_{j=1}^K (y_j - Q(s_j, a_j; \theta))^2$ 
20:     Perform Gradient Descent step on  $L$  to update main network weights  $\theta$ 
21:     if  $t \bmod C = 0$  then
22:       Update Target Network:  $\hat{\theta} \leftarrow \theta$ 
23:     end if
24:     Update State:  $s_t \leftarrow s_{t+1}$ 
25:   end for
26:   Decay Epsilon:  $\epsilon \leftarrow \max(\epsilon_{min}, \epsilon \cdot \epsilon_{decay})$ 
27: end for

```

The agent's policy is guided by an ϵ -**greedy strategy**, where it enhance by selecting a random action with probability ϵ and "exploits" its learned knowledge by choosing the action with the highest Q-value with probability $1 - \epsilon$. The value of ϵ is decayed over time from 1.0 to a small minimum (e.g., 0.01) to shift the agent from exploration to exploitation.

3.7 Performance Evaluation Metric

To conduct a comprehensive evaluation of the three control strategies, a set of diverse performance metrics was chosen. These metrics cover both traffic efficiency and environmental impact, which are extracted directly from the SUMO simulation output.

1. **Average and maximum waiting time (s)**: IT is derived from SUMO's per-vehicle waiting time; captures the level of congestion perceived by drivers, and highlights worst-case delay.
2. **Average and maximum speed (m/s)**: Computed from vehicle trajectories to show the network's overall fluidity.
3. **Congestion index**: Indicate share of lane samples in which vehicles are queueing (waiting time > 5s); exposes imbalanced phases or recurring bottlenecks.
4. **Emission and fuel-consumption totals (mg/ml)**: Summed from SUMO's emission model. These quantify environmental impact, supporting sustainability comparisons.
5. **Lane occupancy statistics**: averages of induction-loop occupancy and their variance, indicating how evenly traffic is distributed across approaches.

3.8 Visualization Dashboard

A Streamlit-based dashboard serves as the user interface for configuring simulations and reviewing their outcomes. The dashboard workflow consists of:

- **Experiment Setup:** The user selects a control strategy, launches the SUMO-backed simulation, and the system spawns a Python subprocess that runs the scenario for the chosen duration.
- **Upon completion,** the simulator stores the aggregated metrics and raw logs to JSON/CSV.
- **Interactive Analysis:** Streamlit loads those results, presents key metrics as KPI cards, and renders comparison charts (e.g., waiting time, throughput) using Plotly. The interface also lists historical runs, allowing side-by-side evaluation of multiple strategies executed earlier in the session.

This interactive interface enhances understanding of the controller's performance and provides a foundation for potential field deployment testing.

Chapter 4

Results

4.1 section a

4.2

Table [4.1](#)

Table 4.1	
	()
Result	

4.3 Summary

Chapter 5

Discussion

Chapter 6

Conclusions

References

- [1] R. Qiu, J. Xu, R. Ke, Z. Zeng, and Y. Wang, "Carbon pricing initiatives based bi-level pollution routing problem," *European Journal of Operational Research*, vol. 286, no. 1, pp. 203–217, 2020.
- [2] D. Zhong, P. Sun, and A. Boukerche, "Empirical study and analysis of the impact of traffic flow control at road intersections on vehicle energy consumption," in *Proceedings of the 18th ACM Symposium on Mobility Management and Wireless Access*. ACM, Nov. 2020, pp. 21–28.
- [3] S. S. Mousavi, M. Schukat, and E. Howley, "Traffic light control using deep policy-gradient and value-function-based reinforcement learning," *IET Intelligent Transport Systems*, vol. 11, no. 7, pp. 417–423, 2017.
- [4] M. Noaeen *et al.*, "Reinforcement learning in urban network traffic signal control: A systematic literature review," *Expert Systems with Applications*, vol. 199, p. 116830, Aug. 2022.
- [5] J. Ault and G. Sharon, "Reinforcement learning benchmarks for traffic signal control," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021.
- [6] M. Ramezani, J. Haddad, and N. Geroliminis, "Dynamics of heterogeneity in urban networks: aggregated traffic modeling and hierarchical control," *Transportation Research Part B: Methodological*, vol. 74, pp. 1–19, 2015.

- [7] I. Arel, C. Liu, T. Urbanik, and A. G. Kohls, "Reinforcement learning-based multi-agent system for network traffic signal control," *IET Intelligent Transport Systems*, vol. 4, no. 2, pp. 128–135, 2010.
- [8] T. Chu, S. Qu, and J. Wang, "Large-scale traffic grid signal control with regional reinforcement learning," in *2016 american control conference (acc)*. IEEE, 2016, pp. 815–820.
- [9] W. Genders and S. Razavi, "Using a deep reinforcement learning agent for traffic signal control," *arXiv preprint arXiv:1611.01142*, 2016.
- [10] K. Shaaban and I. Kim, "Comparison of simtraffic and vissim microscopic traffic simulation tools in modeling roundabouts," *Procedia Computer Science*, vol. 52, pp. 43–50, 2015, the 6th International Conference on Ambient Systems, Networks and Technologies (ANT-2015), the 5th International Conference on Sustainable Energy Information Technology (SEIT-2015). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050915008169>
- [11] D. Krajzewicz, G. Hertkorn, C. Rössel, and P. Wagner, "Sumo (simulation of urban mobility) – an open-source traffic simulation," in *Proceedings of the 4th Middle East Symposium on Simulation and Modelling (MESM2002)*, 2002, pp. 183–187.
- [12] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 177–195, 2013.
- [13] C. Gershenson, "Self-organizing traffic lights," *arXiv preprint nlin/0411066*, 2004.
- [14] L. Singh, S. Tripathi, and H. Arora, "Time optimization for traffic signal control using genetic algorithm," *International Journal of Recent Trends in Engineering*, vol. 2, no. 2, p. 4, 2009.
- [15] T. Chu, J. Wang, L. Codecà, and Z. Li, "Multi-agent deep reinforcement learning

- for large-scale traffic signal control,” *IEEE transactions on intelligent transportation systems*, vol. 21, no. 3, pp. 1086–1095, 2019.
- [16] M. Zhu, Y. Wang, Z. Pu, J. Hu, X. Wang, and R. Ke, “Safe, efficient, and comfortable velocity control based on reinforcement learning for autonomous driving,” *Transportation Research Part C: Emerging Technologies*, vol. 117, p. 102662, 2020.
- [17] J. Casas, J. L. Ferrer, D. Garcia, J. Perarnau, and A. Torday, “Traffic simulation with aimsun,” in *Fundamentals of Traffic Simulation*. Springer, 2010, pp. 173–232.
- [18] A. Stevanovic and M. Zlatkovic, “Evaluation of insync adaptive traffic signal control in microsimulation environment,” in *92nd TRB Annual Meeting, Transportation Research Board*, 2012, conference paper. [Online]. Available: https://www.researchgate.net/publication/274137363_Evaluation_of_InSync_Adaptive_Traffic_Signal_Control_in_Microsimulation_Environment
- [19] S. F. Smith, G. J. Barlow, C. T. Hendrickson et al., “Smart urban signal networks: Initial application of the surtrac adaptive traffic signal control system,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2013. [Online]. Available: <https://aaai.org/papers/00434-13594-smart-urban-signal-networks-initial-application-of-the-surtrac-adaptive-traffic-signal-c>
- [20] D. F. Borges, E. M. Moreira, A. D. de Souza, and J. P. R. Leite, “Traffic light control and machine learning: A systematic mapping review,” in *Proceedings of the 18th International Conference on Information Technology – New Generations (ITNG)*. Cham, Switzerland: Springer, 2021, pp. 11–17.
- [21] S. M. A. B. Al Islam, H. M. A. Aziz, H. Wang, and S. E. Young, “Minimizing energy consumption from connected signalized intersections by reinforcement learning,” in *Proceedings of the 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, Nov. 2018, pp. 1870–1875.

- [22] M. Aslani, M. S. Mesgari, and M. Wiering, "Adaptive traffic signal control with actor-critic methods in a real-world traffic network with different traffic disruption events," *Transportation Research Part C: Emerging Technologies*, vol. 85, pp. 732–752, Dec. 2017.
- [23] E. Van der Pol and F. A. Oliehoek, "Coordinated deep reinforcement learners for traffic light control," *Proceedings of learning, inference and control of multi-agent systems (at NIPS 2016)*, vol. 8, pp. 21–38, 2016.
- [24] F. Xiao, Z. Li, B. Kolosz, C. Chen, H. Wang, M. Barthelemy, and M. Li, "Advances in reinforcement learning for traffic signal control: a review of recent progress," *Intelligent Transportation Infrastructure*, vol. 3, p. liaf009, 2024.
- [25] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [26] M. Wiering, J. v. Veenen, J. Vreeken, A. Koopman *et al.*, "Intelligent traffic light control," 2004.
- [27] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [28] L. Li, Y. Lv, and F.-Y. Wang, "Traffic signal timing via deep reinforcement learning," *IEEE/CAA Journal of Automatica Sinica*, vol. 3, no. 3, pp. 247–254, 2016.
- [29] S. Lange and M. Riedmiller, "Deep auto-encoder neural networks in reinforcement learning," in *The 2010 international joint conference on neural networks (IJCNN)*. IEEE, 2010, pp. 1–8.
- [30] G. Ma, Y. Li, L. Qin, J. Zhou, and X. Yang, "Max-pressure controller for traffic

- networks considering the phase switching loss," *Sustainability*, vol. 17, no. 10, p. 4492, 2024.
- [31] J. C. Burguillo-Rial, P. S. Rodríguez-Hernández, E. Costa-Montenegro, and F. Gil-Castiñeira, "History-based self-organizing traffic lights," *Computing and Informatics*, vol. 28, no. 2, pp. 157–176, 2010.
- [32] K. L. Tan, A. Sharma, and S. Sarkar, "Robust deep reinforcement learning for traffic signal control," *Journal of Big Data Analytics in Transportation*, vol. 2, no. 3, pp. 263–274, 2020.
- [33] Y. Wu, H. Tan, L. Qin, and B. Ran, "Differential variable speed limits control for freeway recurrent bottlenecks via deep actor-critic algorithm," *Transportation research part C: emerging technologies*, vol. 117, p. 102649, 2020.
- [34] Y. Han, M. Wang, L. Li, C. Roncoli, J. Gao, and P. Liu, "A physics-informed reinforcement learning-based strategy for local and coordinated ramp metering," *Transportation Research Part C: Emerging Technologies*, vol. 137, p. 103584, 2022.
- [35] K. Jang, E. Vinitsky, B. Chalaki, B. Remer, L. Beaver, A. A. Malikopoulos, and A. Bayen, "Simulation to scaled city: zero-shot policy transfer for traffic control via autonomous vehicles," in *Proceedings of the 10th ACM/IEEE international conference on cyber-physical systems*, 2019, pp. 291–300.
- [36] B. Chalaki, L. E. Beaver, B. Remer, K. Jang, E. Vinitsky, A. M. Bayen, and A. A. Malikopoulos, "Zero-shot autonomous vehicle policy transfer: From simulation to real-world via adversarial learning," in *2020 IEEE 16th international conference on control & automation (ICCA)*. IEEE, 2020, pp. 35–40.

Appendix A

A

$$\|\mathbf{Ax} - \mathbf{b}\|_2^2 = (\mathbf{Ax} - \mathbf{b})^T (\mathbf{Ax} - \mathbf{b})$$